

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO1 ASSESSMENT TOOL 1 – Case Study Analysis

Course Code:	CSA10 / CSA1063	Course Title:	Software Engineering
CO Assessed:	CO1	Assessment:	Assessment Tool 1 – Case Study Analysis
Weightage:	20%	Total Marks:	25
CO1 Statement:	Apply advanced methodologies and agile project management techniques to manage software projects effectively		
Student Name:	Leo Leenas A	Reg. No.:	192511068
Date:	09 October 2026	Duration:	60 minutes

Code of Conduct

I Leo Leenas A (192511068) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.
Signature of the Student: _____

Annexure A – Case Study Analysis

Instructions to Students:

Each student will be allotted **one problem statement**. Analyze the assigned problem systematically by following the steps given below. Your report should demonstrate your understanding of software engineering concepts and your ability to design an appropriate software solution.

Based on your allotted problem statement, prepare a report by answering the following questions:

- 1. Understand the Problem Statement**
 - Explain the problem in your own words.
 - Identify the objectives of the proposed system.
 - State the scope of the problem.
- 2. Identify Key Stakeholders and Challenges**
 - Identify all the stakeholders involved in the system.
 - Explain the roles and responsibilities of each stakeholder.
 - Discuss the major challenges and issues faced by the stakeholders.
- 3. Analyze the Current Approach**
 - Describe the existing system or current process.
 - Identify its strengths and limitations.
 - Analyze the problems or gaps that need to be addressed.
- 4. Propose a Suitable Solution Using Software Engineering Principles**
 - Design a software-based solution for the given problem.
 - Describe the major functional modules of the proposed system.
 - Identify the functional and non-functional requirements.
 - Explain how software engineering principles such as modularity, scalability, maintainability, reliability, usability, and security are incorporated into your solution.
- 5. Justify the Chosen Methodology/Framework**

- Select an appropriate Software Development Life Cycle (SDLC) model or software development framework (e.g., Agile, Scrum, Waterfall, Spiral, DevOps).
- Justify why the selected methodology/framework is suitable for the proposed system.
- Explain how it supports successful project development and maintenance.

6. Discuss Expected Outcomes and Limitations

- Describe the expected benefits of the proposed solution.
- Explain how the solution addresses the identified challenges.
- Discuss potential risks, implementation challenges, and limitations.
- Suggest possible future enhancements.

Rubrics for Calculation

Criteria	Excellent (5 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Problem Understanding (30%)	Clearly explains the problem, objectives, and scope with complete understanding.	Explains the problem with minor omissions.	Basic understanding with limited explanation.	Poor understanding; objectives and scope are unclear or missing.
Stakeholder & Challenge Analysis (25%)	Identifies all stakeholders, their roles, and challenges with clear analysis.	Identifies most stakeholders and challenges with adequate analysis.	Identifies only a few stakeholders or challenges.	Stakeholders and system analysis are incomplete or inaccurate.
Current System Analysis (20%)	Thoroughly analyzes the existing system, strengths, weaknesses, and identifies all gaps.	Good analysis with most strengths and weaknesses identified.	Limited analysis with partial identification of issues.	Proposed solution is unclear or inappropriate; requirements are largely missing.
Proposed Software Solution & Software Engineering Principles (15%)	Proposes an innovative, feasible solution applying appropriate software engineering principles (modularity, scalability, maintainability, security, etc.).	Suitable solution with good application of software engineering principles.	Basic solution with limited application of software engineering principles.	Methodology is inappropriate, poorly justified, or not discussed.
Methodology/ Framework Justification	Selects the most suitable SDLC/model/fram	Suitable methodology selected with	Methodology selected but justification is	Outcomes, limitations, or future enhancements

(10%)	ework and provides strong technical justification.	adequate justification.	weak.	are missing; report lacks organization and clarity.

Criteria	Mark
Problem Understanding (30%)	/5
Stakeholder & Challenge Analysis (25%)	/5
Current System Analysis (20%)	/5
Proposed Software Solution & Software Engineering Principles (15%)	/5
Methodology/ Framework Justification (10%)	/5
TOTAL	/25

ONLINE BANKING SERVICE MANAGEMENT SYSTEM

Software Engineering - CO1 Assessment Tool 1: Case Study Analysis

Question No.: 18	Register No.: 192511068
Student Name: Leo Leenas A	Date: 09 October 2026

Assigned Problem Statement

Online Banking Service Management System: Develop a secure platform for customers to view accounts, transfer funds, pay bills, download statements, and receive transaction notifications.

1. Understand the Problem Statement

The Online Banking Service Management System is a secure web-based application that lets an authorized customer complete common banking tasks without having to visit a branch for every service. A customer can sign in, check account balances and transaction history, transfer funds, pay supported bills, download account statements, and receive alerts about important account activity. The system must protect confidential information and make sure that every transaction is recorded accurately.

Objectives

- Provide convenient access to account information at any time, subject to service availability.
- Allow customers to transfer money and pay supported bills through a verified, traceable process.
- Provide downloadable statements and a searchable transaction history.
- Send timely notifications for transfers, bill payments, logins, and other selected account events.
- Reduce avoidable branch visits and routine staff workload while maintaining strong security and audit records.

Scope

The proposed system covers customer authentication, account viewing, beneficiary management, fund transfers, bill payments through approved payment services, statement generation, transaction notifications, and authorized administrative monitoring. It is intended to connect to the bank's existing core banking and payment systems through controlled interfaces; it is not a replacement for the bank's entire core banking platform. Cash deposits, ATM hardware, loan approval, and other services outside the assigned problem are not part of the initial version, though some could be considered later.

2. Identify Key Stakeholders and Challenges

Stakeholders, roles, and challenges

- Customers: View balances, review transactions, transfer funds, pay bills, and download statements. Their main concerns are account safety, ease of use, transaction delays, forgotten credentials, and uncertainty when a transaction is pending.
- Bank operations and customer-support staff: Assist customers, investigate complaints, verify transaction issues, and support account-service requests. They need accurate records and clear transaction statuses, but may face high enquiry volumes and incomplete information from customers.
- Bank administrators: Manage user access, approved service settings, operational reports, and exception handling within their permissions. They must avoid unauthorized access and keep administrative actions traceable.
- IT, security, and maintenance team: Operate the application, protect it from attacks, monitor performance, manage backups, fix defects, and deploy updates. Their challenges include evolving cyber threats, integration failures, service availability, and safe software changes.
- Payment gateways and biller services: Process supported bill payments and return payment status. Their availability, response delays, and differing interface requirements can affect the customer experience.

- Bank management, compliance, and audit personnel: Ensure that the service meets bank policy and applicable legal or regulatory requirements, and that transactions can be reviewed when needed. They require clear audit trails, privacy safeguards, and reliable reports.

The interests of these stakeholders can conflict. Customers expect fast, simple service, while the bank must verify identity, prevent fraud, maintain accurate records, and follow its approval controls. The design should balance usability with security rather than weakening checks to make a transaction quicker.

3. Analyze the Current Approach

In a typical traditional or fragmented banking process, customers may use a branch counter, ATM, telephone support, paper statements, or separate digital channels depending on the service. Staff may need to look up records in internal systems to answer enquiries or investigate a disputed payment. This is a general analysis of the likely current process; the case study does not identify a particular bank or its existing technology.

Strengths

- Branch staff can provide direct assistance for complex cases and customers who need support.
- Established banking procedures, authorization rules, and transaction records provide a basis for controlled processing.
- Receipts and statements can help customers review their activity and raise a query.

Limitations and gaps

- Branch-based requests can involve travel, waiting, and dependence on branch opening hours.
- Using separate channels for different services can make the customer experience inconsistent and transaction tracking harder.
- Manual data entry or repeated information handling can cause mistakes and increase the time needed to resolve issues.
- Customers may not receive a clear, timely status for a transfer or bill payment, especially when an external service is delayed.
- Paper-based statement handling is less convenient for searching, storing, and downloading records.
- Without well-designed security alerts and access controls, customers and the bank may have less visibility into suspicious activity.

The main gap is the lack of one convenient and consistently monitored channel for the required services. The proposed platform addresses this by bringing account viewing, payments, statements, and notifications into one authenticated interface and by showing transaction status clearly.

4. Propose a Suitable Solution Using Software Engineering Principles

Proposed system design

The solution is a responsive web application built with a client-server architecture. The customer uses a browser interface. A secure application/API layer validates requests and applies business rules. Separate services handle authentication, accounts, transfers, bill payments, statements, and notifications. The platform connects only to approved bank core systems and payment/biller interfaces. A protected database stores application data and audit records, while monitoring and backup services support safe operation. Access to account data is checked on the server for every request; hiding a screen element alone is not an adequate security control.

Major functional modules

- Customer authentication and profile: Sign-in, multi-factor verification where required, session timeout, profile settings, secure sign-out, and controlled account recovery.
- Account dashboard: Show the customer's permitted accounts, available balance, recent transactions, and transaction details. A user must never be able to view another customer's account by changing a request or URL.
- Beneficiary and fund transfer: Add or manage beneficiaries under bank rules, enter transfer details, confirm the recipient and amount, perform authorization checks, submit the transaction, and show a reference number and final or pending status.

- Bill payment: Display supported billers, collect required bill details, confirm the amount, submit payment through an approved service, and record its status and reference.
- Statement management: Search or filter transaction records by date and type, generate statements for a selected period, and download them in a common format such as PDF.
- Notification service: Send transaction confirmations and relevant security alerts through configured channels such as email, SMS, or in-app notifications. Notifications should not expose full account credentials or sensitive authentication secrets.
- Administration, monitoring, and audit: Allow authorized staff to manage roles and service settings, review exceptions, monitor system health, and inspect audit logs. Administrative privileges should be limited to the tasks each role needs.

Functional requirements

- FR1: The system shall authenticate customers and end sessions securely after sign-out or an appropriate period of inactivity.
- FR2: The system shall display only accounts and transactions authorized for the signed-in customer.
- FR3: The system shall let an eligible customer initiate a transfer after validating the recipient, amount, available funds, limits, and required authorization.
- FR4: The system shall submit supported bill payments, preserve a transaction reference, and display confirmed, pending, or failed status accurately.
- FR5: The system shall generate and allow customers to download statements for a selected account and period.
- FR6: The system shall create configured transaction and security notifications and record the delivery result where available.
- FR7: Authorized administrators shall be able to review permitted operational reports and audit records without unrestricted access to customer secrets.

Non-functional requirements

- Security and privacy: Encrypt data in transit, protect stored sensitive data, use strong password handling and multi-factor authentication where appropriate, enforce role-based access, and test for common web vulnerabilities.
- Reliability and integrity: A transfer must be processed consistently. Debit and credit updates should not be partially committed; unique transaction identifiers, duplicate-request protection, reconciliation, and clear status handling should reduce double processing and ambiguity.
- Performance and availability: Common account-viewing tasks should respond promptly under expected load, and monitoring should detect service failures. The service should have planned recovery procedures for outages.
- Usability and accessibility: Use clear labels, simple navigation, readable messages, validation near input fields, and layouts that work on desktop and mobile browsers.
- Scalability: The service should allow capacity to be increased as customer numbers and transaction volume grow.
- Maintainability and testability: Keep components separated, document interfaces, automate repeatable tests, and use controlled versioning and deployment practices.
- Auditability: Record key security and transaction events with timestamps and references, while protecting the logs from unauthorized alteration and avoiding unnecessary sensitive data.

Application of software engineering principles

- Modularity: Each major function is placed in a separate module with a defined interface. A change to statements should not require rewriting the transfer module.
- Scalability: The web layer and service components can be scaled according to usage, while database and transaction capacity are planned and monitored.
- Maintainability: Consistent coding standards, documentation, automated tests, code reviews, and controlled releases make faults easier to locate and fix.
- Reliability: Validate inputs, use safe transaction processing, log failures, reconcile payment statuses, test recovery, and maintain backups.

- Usability: Use predictable navigation, clear confirmations, accessible layouts, and understandable error messages so customers can complete tasks correctly.
- Security: Apply least privilege, server-side authorization, encryption, secure sessions, multi-factor checks, audit logging, vulnerability testing, and protection against repeated or unauthorized requests.

5. Justify the Chosen Methodology/Framework

Agile development using the Scrum framework is suitable for this proposed system. The customer interface, bank integrations, security controls, and notification behaviour need testing and feedback as they are developed. Scrum divides work into short, planned sprints, producing a reviewable increment at the end of each sprint. This gives the team opportunities to find usability problems and integration risks early instead of waiting until the entire system is finished.

How Scrum would be applied

- Product backlog: Maintain and prioritize requirements for login, account viewing, transfers, bill payments, statements, notifications, administration, security, and operational monitoring.
- Sprint planning and daily coordination: Agree on a realistic sprint goal, assign work, and identify blockers or integration dependencies.
- Incremental implementation: Begin with requirements and architecture, then build authentication and the account dashboard, followed by transfers and bill payments, and then statements, notifications, and administrative monitoring.
- Testing and review: Run unit, integration, security, regression, and user-acceptance tests as features are added. Demonstrate each increment to relevant stakeholders and collect feedback.
- Retrospective and maintenance: Review how the team worked, improve the process, fix defects, monitor production, and prioritize safe enhancements in later sprints.

Agile is not a reason to skip financial controls or release untested changes. Each increment should meet agreed acceptance criteria, pass required security and transaction tests, receive appropriate review, and be released through a controlled approval process. Scrum supports successful development by improving visibility, feedback, and adaptability; ongoing monitoring, patching, backups, and regression testing support maintenance after deployment.

6. Discuss Expected Outcomes and Limitations

Expected outcomes and how challenges are addressed

- Convenience: Customers can view accounts, transfer funds, pay supported bills, and download statements through one online interface, reducing avoidable branch visits.
- Better transaction visibility: Reference numbers, clear statuses, transaction history, and notifications help customers understand what happened and when further checking may be needed.
- Fewer routine manual tasks: Automated statements and digital requests can reduce repetitive work for bank staff and make records easier to retrieve.
- Improved control: Authentication, server-side permission checks, protected audit trails, and transaction validation reduce the chance of unauthorized access and untraceable changes.
- More informed support: Staff with appropriate access can use transaction references and exception logs to investigate reports more consistently.

Risks, implementation challenges, and limitations

- Cyberattacks, phishing, and credential theft: Use multi-factor verification, secure sessions, monitoring, regular security testing, customer awareness messages, and a clear incident-response process.
- Failed, delayed, or duplicate transactions: Use unique references, duplicate-request protection, atomic transaction handling where supported, status reconciliation, and clear pending/failed messages. Do not tell a customer that a payment succeeded before confirmation is received.
- External integration or service downtime: Design safe timeouts and retries, monitor interfaces, keep audit records, and provide a status-check or support path when a provider response is uncertain.

- Data privacy and access mistakes: Apply least privilege, review access regularly, protect logs and backups, and test that a customer cannot access another customer's data.
- Internet dependence and digital-access barriers: Customers need a reliable connection and basic digital skills; accessible design and existing assisted banking channels remain important alternatives.
- Delivery cost and complexity: Connecting legacy bank services, testing payment flows, and meeting internal controls may take substantial time and specialist effort. The system also cannot guarantee that every external biller is always available.

Future enhancements

- A dedicated mobile application with secure device registration and carefully implemented biometric sign-in.
- Fraud-risk monitoring that flags unusual activity for review without automatically treating every alert as confirmed fraud.
- Multilingual support, improved accessibility options, and customer-selected notification preferences.
- Optional spending summaries and budgeting insights based on customer-authorized transaction data.
- Additional supported billers and payment services after security, compatibility, and operational reviews.

Conclusion

The Online Banking Service Management System provides a practical way to bring essential customer banking services into one secure web platform. A modular design separates account viewing, transfers, bill payment, statements, and notifications, while access controls, transaction validation, audit records, and recovery planning protect the service. Agile/Scrum supports staged development and regular stakeholder feedback, provided every increment still meets the bank's security, testing, and release requirements. The result should improve convenience and transaction visibility while recognizing that internet access, external-service availability, and ongoing security maintenance remain important constraints.